

Ce TP sera à réaliser sur le remotelabz, dans la lab INFO_0702_TP. Les sections 1 et 2 seront à faire sur la machine `debian_info0702_tp`, tandis que la section 3 sera à faire sur les machines `worker_info_0702_tp` et `manager_info_0702_tp`.

Sur toutes la Debian, vous avez un utilisateur avec accès à `sudo` qui s'appelle `info0702` dont le mot de passe est `info0702`. Sur les `worker` et `manager`, le nom de l'utilisateur et son mot de passe sont affichés au démarrage.

1 *Namespaces*

- Dans cette première série d'exercices, sauf mention contraire, les commandes utilisant `unshare` devront être exécutées en tant que `root`.
- Par convention, `ns_root` sera votre *namespace* de base, `nsx` seront les différents *namespaces* que vous créerez.
- Si pour une raison ou une autre l'affichage du `$BASHPID` est vide, vous afficherez `$$` à la place.
- À partir du moment où vous avez `nsx` de présent et actif, s'il est demandé de faire des commandes dans `ns_root`, il faudra utiliser un autre terminal.
- Vous vous déconnecterez de tous vos *namespaces* entre deux exercices.

1.1 *Namespace UTS*

- 1 Affichez le *hostname* de votre `ns_root`.
- 2 Faites un *namespace* de type UTS.
- 3 Affichez le *hostname* de `ns1`.
- 4 Changez le *hostname* de `ns1`.
- 5 Vérifiez le *hostname* de `ns1`.
- 6 Vérifiez que le *hostname* de `ns_root` n'a pas été modifié.
- 7 Affichez le `$BASHPID` dans `ns1`.
- 8 Entrez dans `ns1` depuis `ns_root`.
- 9 Vérifiez le bon fonctionnement en affichant le *hostname*.
- 10 Déconnectez-vous de `ns1` (Ctrl+D) pour revenir à `ns_root` (tout en conservant un second terminal dans `ns1`).
- 11 Affichez tous les *namespaces* de type UTS.

1.2 *Namespace PID*

- 12 Faites un *namespace* de type PID avec un `fork` et en montant le `/proc/ns`.
- 13 Vérifiez que le `$BASHPID` vaut 1.
- 14 Dans `ns1`, affichez les *namespaces* de type PID.
- 15 Dans `ns_root`, affichez les *namespaces* de type PID

1. avec votre *user* habituel
2. avec `root`

y a-t-il une différence ?

- 16 Les PID dans `ns1` et `ns_root` sont-ils les mêmes ?

1.3 *User-namespace*

- 17 En tant qu'utilisateur normal, faites un *user-namespace* en le montant comme un utilisateur `root`.
- 18 Dans `ns1`, vérifiez que vous êtes bien `root` avec `id`.

- 19 Affichez le contenu de `/home`. Notamment à qui et quel groupe appartiennent les répertoires présents dedans.
- 20 Essayez d'aller dans `/root`.
- 21 Affichez à qui et quel groupe appartient `/root`.
- 22 Expliquez pourquoi vous ne pouvez pas y aller.
- 23 Déconnectez-vous de `ns1`.
- 24 En tant qu'utilisateur normal, faites un *user-namespace* sans être monté en tant que `root`.
- 25 Vérifiez votre identifiant.
- 26 Affichez les détails de `/home`.
- 27 Pouvez-vous aller dans `/home/<user>` ?
- 28 Affichez les détails de `/root`.
- 29 Vérifiez que vous êtes le même utilisateur et dans le même groupe que `/root`.
- 30 Allez dans `/root`.
- 31 Pourquoi n'avez-vous pas pu y accéder ?
- 32 Récupérez le `$BASHPID` de `ns1`.
- 33 Dans `ns_root`, faites un `echo` (en tant que `root`) de `"0 1000 1"`, dans `/proc/<PID>/uid_map`.
- 34 Dans `ns1`, vérifiez votre identité.
- 35 Dans `ns_root`, refaites le `echo`, mais avec `"1000 1000 1"`.
- 36 Expliquez l'erreur obtenue.
- 37 Déconnectez-vous de `ns1`.
- 38 En tant qu'utilisateur normal, faites un *namespace* de type UTS.
- 39 Avez-vous pu le créer ?
- 40 En tant qu'utilisateur normal, faites un *user-namespace* en vous montant comme `root`.
- 41 Changez le *hostname* de `ns1`.
- 42 Avez-vous pu le changer ?
- 43 Faites un *namespace* de type UTS.
- 44 Changez le *hostname* de `ns2`.

1.4 Mount-namespaces

- 45 En tant qu'utilisateur normal, faites un *user-namespace* en vous montant en tant que `root`. Vous utiliserez l'option `--mount` pour qu'il soit aussi un *mount-namespace*.
- 46 Créez un répertoire dans `ns1`.
- 47 Montez ce répertoire avec `mount -t tmpfs none <dir_name>`.
- 48 Vérifiez qu'il est bien monté.
- 49 Vérifiez qu'il n'est pas monté dans `ns_root`.
- 50 Démontez le répertoire et supprimez-le dans `ns1`.
- 51 Déconnectez-vous de `ns1`.
- 52 Faites un nouveau répertoire, et mettez-y un fichier.
- 53 En tant qu'utilisateur normal, faites un *user-namespace* en vous montant en tant que `root` et avec l'option de *mount-namespace*.
- 54 Dans `ns1`, affichez le contenu du répertoire récemment créé.
- 55 Dans `ns1`, montez le répertoire comme précédemment.
- 56 Vérifiez que rien n'est visible dans ce répertoire.
- 57 Faites un *user-namespace* en vous montant en tant que `root`.
- 58 Affichez les volumes montés.
- 59 Démontez votre répertoire monté.
- 60 Pourquoi ne pouvez-vous pas le démonter ?

2 cgroups

- 1 Installez les paquets `stress` et `htop`.
- 2 Dans un terminal, lancez `htop`, et gardez-le ouvert.
- 3 Dans un autre terminal, exécutez la commande `stress --cpu 8 --io 4 --vm 2 --vm-bytes 128M--timeout 10s`. Vérifiez dans `htop` que tous vos cœurs sont bien utilisés à 100 %.
- 4 Créez un service `systemd` qui exécutera `stress` avec la même commande.

- 5 Dans le terminal où tourne **htop**, arrêtez-le et démarrez **systemd-cgtop**.
- 6 Démarrez le service nouvellement créé, et vérifiez qu'il apparaît bien dans **systemd-cgtop**.
- 7 Vérifiez que les fichiers liés au cgroup du service ne sont présents que pendant qu'il est lancé.
- 8 Modifiez le service pour qu'il ne puisse prendre que 50 % du CPU.
- 9 Relancez le service et vérifiez son statut.
- 10 Modifiez le service pour qu'il exécute la commande **stress** avec la moitié de vos cœurs uniquement (**--cpu**).
- 11 Relancez le service et vérifiez son statut.
- 12 Vous pouvez supprimer votre service ainsi que les paquets **stress** et **htop**.

3 Docker

3.1 Commandes de base

- 1 Exécutez l'image **hello-world** pour vérifier le bon fonctionnement de votre Docker.
- 2 Affichez les différentes images présentes sur votre machine.
- 3 Affichez les conteneurs en cours d'exécution.
- 4 Affichez tous les conteneurs présents.
- 5 Purgez vos conteneurs.
- 6 Supprimez l'image **hello-world**.

3.2 Redirection de port et de volumes

- 7 Stoppez tous serveurs Web susceptibles de tourner sur votre machine.
- 8 Exécutez l'image **nginx** en mode détaché.
- 9 Essayez d'afficher la page Web **http://localhost**.
- 10 Stoppez le conteneur.
- 11 Exécutez l'image **nginx** en mode détaché et faites une redirection du port 8080 de votre machine vers le port 80 du *guest*.
- 12 Accédez à la page Web **http://localhost:8080**.
- 13 Stoppez le conteneur.
- 14 Faites un répertoire **www** contenant une page Web.
- 15 Exécutez l'image **nginx** en mode détaché et faites une redirection du port 8080 de votre machine vers le port 80 du *guest* et du volume **\$PWD/www** de votre machine vers **/usr/share/nginx/html** du *guest*.
- 16 Accédez à la page Web **http://localhost:8080**.
- 17 Stoppez le conteneur.
- 18 Purgez vos conteneurs.

3.3 Dockerfile

- 19 Créez un Dockerfile.
- 20 Utilisez une *Debian-like* comme base (peu importe laquelle).
- 21 Utilisez la commande pour mettre à jour la base de données des paquets de l'image, puis installez **cowsay** et **dumb-init**.
- 22 Ajoutez une variable d'environnement qui s'appelle **MESSAGE** et initialisez la avec une chaîne de caractères.
- 23 Faites que votre point d'entrée soit **dumb-init**.
- 24 Exécutez la commande **bash -c /usr/games/cowsay \$MESSAGE**.
- 25 Compilez votre image en lui donnant le tag **cowsay:latest**.
- 26 Vérifiez qu'elle fait bien partie de vos images.
- 27 Exécutez-la en mode interactif et avec l'option qui supprime le conteneur une fois l'exécution terminée.
- 28 Exécutez-la à nouveau en modifiant la variable d'environnement **\$MESSAGE**.
- 29 Dans le Dockerfile, modifiez la variable d'environnement **\$MESSAGE**.
- 30 Reconstituez l'image en lui donnant comme tag **cowsay:new**.

- **31** Vérifiez que vous avez bien vos deux images `cowsay`.
- **32** Exécutez l'image `cowsay:new`.
- **33** Faites une copie de l'image `cowsay:new` vers `cowsay:v1.0`.
- **34** Vérifiez vos images.
- **35** Supprimez l'image `cowsay:latest`.

3.4 Docker Compose

Vous utiliserez le répertoire `app` présent sur la Debian de base qui contient un script Python. Ce répertoire est dans le `home` d'`info0702`.

- **36** Installez Docker Compose sur votre machine.
- **37** Écrivez un fichier `requirements.txt` qui contient les deux modules Python à installer.
- **38** Créez un `Dockerfile`.
- **39** La base sera `python:3.10-alpine`.
- **40** Le répertoire de travail s'appellera `/code`.
- **41** Ajoutez les deux variables d'environnement

`FLASK_APP` qui est le nom de l'application

`FLASK_RUN_HOST` qui est l'adresse d'écoute (0.0.0.0 pour ANY)

- **42** Installez les dépendances `gcc`, `musl-dev` et `linux-headers` avec la commande `apk add --no-cache <paquets>`.
- **43** Faites une copie de votre répertoire courant.
- **44** Installez le contenu des *requirements* avec `pip install -r <fichier>`.
- **45** Partagez le port 5000.
- **46** Lancez la commande `flask run --debug`.
- **47** Créez un fichier `compose.yml`.
- **48** Le contenu de ce fichier sera :

`services:`

```
web:
  build: .
  ports:
    - "8000:5000"
redis:
  image: "redis:alpine"
```

- **49** Expliquez le contenu de ce fichier.
- **50** Vérifiez la configuration.
- **51** Lancez cette configuration.
- **52** Vérifiez que vous avez accès à `http://localhost:8000` et que le nombre de fois affiché augmente à chaque reconnexion.
- **53** Affichez les images.
- **54** Fermez les conteneurs de votre *compose*.
- **55** Ajoutez un *watcheur* du module `web` dans votre `compose.yml` :

`develop:`

```
watch:
  - action: sync
    path: .
    target: /code
```

- **56** Vérifiez la configuration.
- **57** Lancez votre *compose* en mode surveillance.
- **58** Vérifiez que votre application fonctionne toujours.
- **59** Dans le code Python, modifiez le message affiché.
- **60** Vérifiez la bonne prise en compte par le *watcheur*.
- **61** Vérifiez que le message soit bien changé sur l'application.
- **62** Faites un fichier YAML avec comme `services` le module `redis`.

- **63** Dans votre fichier `compose.yml`, retirez la partie `redis`.
- **64** Au début de `compose.yml`, incluez le nouveau YAML créé.
- **65** Vérifiez que votre application fonctionne toujours.
- **66** Fermez tous vos conteneurs.

3.5 Docker Swarm

Pour cette partie, nous nommerons `worker` la machine `worker_info0702_tp` et `manager` la machine `manager_info0702_tp`. Chacune de ces machines ont un répertoire `swarm` dans le `home` d'`info0702`. Ce sera ce répertoire qui nous servira de base. Je vous laisse juge de savoir s'il faut utiliser celui du `worker`, `manager` ou les deux.

Si `worker` et `manager` ont la même adresse IP, il faudra soit faire un `dhcpcd` à chaque démarrage, soit faire (une seule fois par machine) :

```
$ sudo rm /etc/machine-id
$ sudo systemd-machine-id-setup
$ sudo reboot
```

- **67** Afin de générer les images faites `make docker-images`.
- **68** Sur `manager`, initialisez le `swarm`.
- **69** Vérifiez les nœuds présents dans le `cluster`.
- **70** Sur `worker`, rejoignez le `swarm`.
- **71** Vérifiez les nœuds présents dans le `cluster` sur `worker`.
- **72** Pourquoi vous ne pouvez pas le faire ?
- **73** Vérifiez les nœuds du `cluster` et leurs rôles.
- **74** Sur `worker`, vérifiez que vous n'avez pas de conteneurs provenant du `cluster` en cours d'exécution.
- **75** Sur `manager`, déployez la `stack`.
- **76** Vérifiez sur `worker`, qu'il a bien des conteneurs du `cluster` en cours d'exécution.
- **77** Sur `manager` affichez tous les processus lancés dans le `cluster`.
- **78** Quittez le `swarm` sur `worker`.
- **79** Sur `manager`, affichez les processus lancés dans le `cluster`, ainsi que les nœuds présents.
- **80** Quittez le `swarm` sur `manager`.
- **81** Affichez les nœuds présents dans le `cluster` sur `manager`.
- **82** Pourquoi vous ne pouvez pas le faire ?